

PRAKTIKUM BASIS DATA

MODUL 11

NoSQL – Key Value Database



**Universitas
Telkom**

Oleh:

Izzan Maula Rifqi

103122430009

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

MODUL 11

JURNAL

1. Davviz adalah seorang Backend Engineer handal gg jago yang sedang membangun platform streaming musik generasi terbaru bernama Spotifine. Aplikasi ini akan memiliki jutaan pengguna aktif, sehingga Davviz membutuhkan database yang tahan banting, dan tidak akan mati meskipun salah satu servernya mengalami gangguan. Davviz ditugaskan untuk merancang database yang menyimpan Data Musisi di dalam platform Spotifine. Karena satu musisi bisa memiliki banyak genre dan pengikut dari berbagai negara, Davviz memilih Cassandra sebagai solusinya.

Berikut atribut yang perlu dibuat :

- a. Artist ID: ID unik untuk setiap musisi (Primary Key).
- b. Artist Name: Nama lengkap musisi atau nama panggung.
- c. Followers: Jumlah pengikut (Tipe data int).
- d. Origin: Negara asal musisi.
- e. Genres: Daftar aliran musik yang diusung (Gunakan tipe data set<text> karena satu musisi bisa memiliki genre lebih dari satu, misal: Pop dan R&B).

2. Implementasi

- a. Buat Keyspace
 - buatlah sebuah Keyspace dengan nama spotifine_0802 sebagai wadah utama data aplikasi
 - Gunakan strategi replikasi SimpleStrategy dengan replication_factor 1.
 - Tuliskan query-nya dan sertakan Screenshot saat keyspace berhasil dibuat dan dipanggil menggunakan perintah USE
- b. Buat Table
 - Buatlah tabel bernama musician_data di dalam keyspace spotifine_0802 dengan atribut yang sudah direncanakan Davviz.
 - Tuliskan query CREATE TABLE sesuai atribut di atas dan sertakan Screenshot.

Jawab:

```
Cassandra CQL Shell
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 2.2.3 | CQL spec 3.3.1 | Native protocol v4]
Use HELP for help.
WARNING: pyreadline dependency missing. Install to enable tab completion.
cqlsh> CREATE keyspace spotifine_8020 with replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE spotifine_8020;
cqlsh:spotifine_8020>
```

```
Cassandra CQL Shell
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 2.2.3 | CQL spec 3.3.1 | Native protocol v4]
Use HELP for help.
WARNING: pyreadline dependency missing. Install to enable tab completion.
cqlsh> CREATE keyspace spotifine_8020 with replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE spotifine_8020;
cqlsh:spotifine_8020> CREATE TABLE musician_data (
...   artist_id text PRIMARY KEY,
...   artist_name text,
...   followers int,
...   origin text,
...   genres set<text>
... );
cqlsh:spotifine_8020>
```

```
Cassandra CQL Shell
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 2.2.3 | CQL spec 3.3.1 | Native protocol v4]
Use HELP for help.
WARNING: pyreadline dependency missing. Install to enable tab completion.
cqlsh> CREATE KEYSPACE spotifine_0802 WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
AlreadyExists: Keyspace 'spotifine_0802' already exists
cqlsh> USE spotifine_0802;
cqlsh:spotifine_0802> CREATE TABLE musician_data (
...     artist_id text PRIMARY KEY,
...     artist_name text,
...     followers int,
...     origin text,
...     genres set<text>
... );
AlreadyExists: Table 'spotifine_0802.musician_data' already exists
cqlsh:spotifine_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres) VALUES ('M001', 'Tulus', 5000000,
'Indonesia', {'Pop', 'Jazz'});
cqlsh:spotifine_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres) VALUES ('M002', 'Bruno Mars', 150
00000, 'USA', {'Pop', 'R&B', 'Funk'});
cqlsh:spotifine_0802> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres) VALUES ('M003', 'Coldplay', 20000
000, 'UK', {'Pop', 'Rock', 'Alternative'});
cqlsh:spotifine_0802> SELECT * FROM musician_data;

artist_id | artist_name | followers | genres | origin
-----+-----+-----+-----+-----
M001 | Tulus | 5000000 | {'Jazz', 'Pop'} | Indonesia
M002 | Bruno Mars | 15000000 | {'Funk', 'Pop', 'R&B'} | USA
M003 | Coldplay | 20000000 | {'Alternative', 'Pop', 'Rock'} | UK

(3 rows)
cqlsh:spotifine_0802>
```

Keuntungan utamanya adalah menghindari penggunaan JOIN dan mempercepat performa read (baca), Karena di RDBMS (Relasional Tradisional) satu musisi bisa memiliki banyak genre (relasi One-to-Many atau Many-to-Many), dan diharuskan membuat tabel baru dan menggunakan operasi JOIN untuk menampilkan profil musisi beserta genre-nya secara lengkap. Operasi JOIN ini sangat berat dan lambat jika pengguna aplikasi mencapai jutaan. Sedangkan di Cassandra dengan tipe data set<text>, kita bisa menyimpan berbagai genre langsung di dalam baris data musisi yang sama atau denormalisasi. Aplikasi seperti Spotifine ini dapat menarik profil musisi beserta seluruh genre-nya hanya dengan satu kali query saja, tanpa perlu melakukan JOIN tabel sama sekali. Ini membuat sistem menjadi sangat ringan dan responsif untuk jutaan pengguna aktif.